

Falando um pouco de projeto de algoritmos...



DIVISÃO E CONQUISTA

Divisão e Conquista

- Método usado para projeto de algoritmos:
  - **Divisão**: Se o tamanho da entrada é muito grande para aplicação de uma solução simples, dividir o problema e dois ou mais subproblemas disjuntos
  - **Conquista**: Usar o método recursivamente para resolver os subproblemas
  - **Combinação**: Tomar as soluções dos subproblemas e combina-las para compor uma solução para o problema original.

Multiplicação de Matrizes(naïve)

- Há algoritmos baseados em divisão em conquista para multiplicar matrizes

Exemplo 1 (ordem n x n)

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

A B C

- Matrizes originais são n x n!
- Cada pequena matriz tem ordem n/2 x n/2

Multiplicação de Matrizes(naive)

- **Divisão**: Se a ordem das matrizes tem $n \geq 2$, dividir a ordem pela metade, gerando 4 matrizes para cada matriz sendo multiplicada, com total de 8 matrizes.
- **Conquista**: Multiplicar as matrizes menores geradas usando a multiplicação de matrizes (ae,bg,af,bh,ce,dg,cf,dh) da figura anterior
- **Combinação**: Realize 4 somas de matrizes resultantes da conquista (ae+bg, af+bg,ce+dg, cf+dh) para gerar o resultado final.

Exemplo 1 (ordem n x n)

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

A B C

$$T(n) = \begin{cases} c & n = 1 \\ 8T\left(\frac{n}{2}\right) + O(n^2) & n > 1 \end{cases}$$

$$\Theta(n^3)$$

* Matrizes são n x n!

Exemplo 2: Strassem

p1 = a(f - h)
p2 = (a + b)h
p3 = (c + d)e
p4 = d(g - e)
p5 = (a + d)(e + h)
p6 = (b - d)(g + h)
p7 = (a - c)(e + f)

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} p5 + p4 - p2 + p6 & p1 + p2 \\ p3 + p4 & p1 + p5 - p3 - p7 \end{bmatrix}$$

$$T(n) = \begin{cases} c & n = 1 \\ 7T\left(\frac{n}{2}\right) + O(n^2) & n > 1 \end{cases}$$

$$\Theta(n^{\log_2 7}) = \Theta(n^{2.8074})$$

* Matrizes são n x n!

Multiplicação de Matrizes

- **Divisão**: Se a ordem das matrizes tem $n \geq 2$, dividir a ordem pela metade, gerando 4 matrizes para cada matriz sendo multiplicada, com total de 8 matrizes (a,b,c,d,e,f,g,h).
- **Conquista**: Computar as 7 multiplicações de matrizes menores descritas no slide anterior (p1 a p7)
- **Combinação**: Realize as somas (p5+p4+p2+p3, p1+p2, p3+p4, p1+p5-p3-p7)

Curiosidades!

- Até 2020, Strassem ainda é o melhor algoritmo em termos práticos para multiplicar matrizes
- Melhor algoritmo teórico foi proposto em 2014.
  - Le Gall, François. "Powers of tensors and fast matrix multiplication." *Proceedings of the 39th international symposium on symbolic and algebraic computation*. 2014.

"the input matrices must be astronomically large for the difference in time to be apparent."

- Seu custo:

$$\Theta(n^{2,373})$$



Ordenação, e
mais alguns exemplos de uso da divisão e conquista

MergeSort

- **Divisão**: Se S tem $n \geq 2$ elementos, dividir seus elementos em duas subsequencias S_1 e S_2 com respectivamente $\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$ elementos.
- **Conquista**: Ordernar as subsequencias S_1 e S_2 usando MergeSort.
- **Combinação**: Intercale os elementos de S_1 e S_2 de forma a obter uma seqüência ordenada

Merge Sort: Algoritmo

```
Merge-Sort(A, p, r)
    if p < r then
        q←(p+r)/2
        Merge-Sort(A, p, q)
        Merge-Sort(A, q+1, r)
        Merge(A, p, q, r)
```

Merge(A, p, q, r)

Retire o menor entre o menor dos elementos de A[p…q] e A[q+1…r] e acrescente ao resultado. Repita até que as duas sub-sequências estejam vazias.

Algoritmos já estudados

- Ordenação por inserção
  - Pior caso: $\Theta(n^2)$
  - Não é necessário espaço adicional
- Merge sort
  - Pior caso: $\Theta(n \log n)$
  - Requer espaço adicional de memória devido a recursão

Quicksort

QuickSort

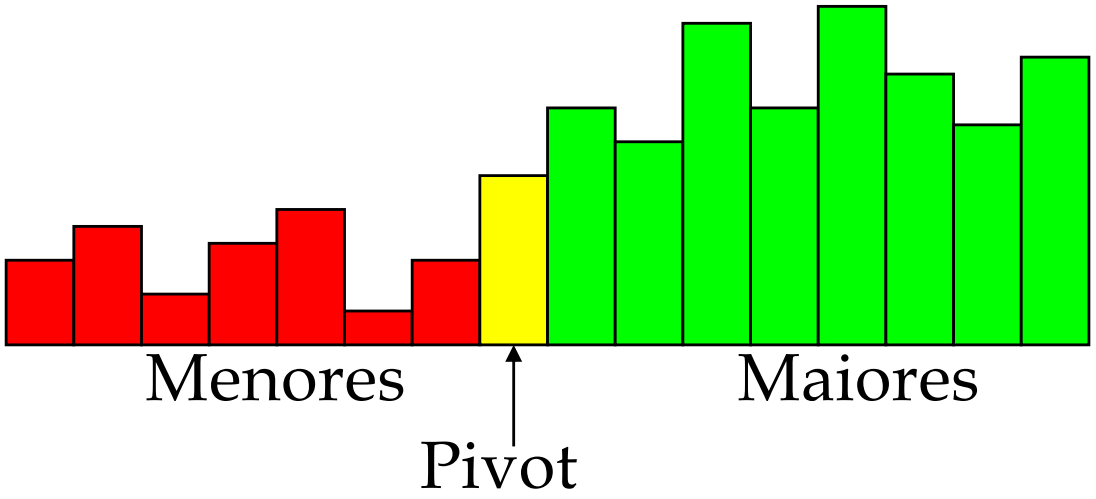
- **Divisão**: Se S tem $n \geq 2$ elementos, escolher um pivot e particionar o conjunto a partir desse pivot, encontrando S_1 com a elementos e S_2 com n-a elementos.
- **Conquista**: Ordernar as subsequencias S_1 e S_2 usando QuickSort.
- **Combinação**: No caso do quicksort, não precisa mais fazer nada (isso acontece em algumas soluções de divisão e conquista!)

Quicksort

- Características
  - ordenação ”in place”, ou seja, a ordenação é feita no próprio arranjo de entrada
  - Método prático:
    - Caso médio: $O(n \log n)$
    - Pior caso: $O(n^2)$

Quicksort

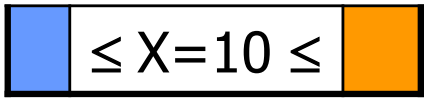
- Divisão e Conquista
  - **Divisão:** particionar o arranjo em dois sub-arranjos de tal forma que todos os elementos no sub-arranjo 1 sejam menores que elementos no sub-arranjo 2
  - **Conquista:** Ordenar recursivamente os sub-arranjos
  - **Combinação:** ordenação trivial é executada "in place"

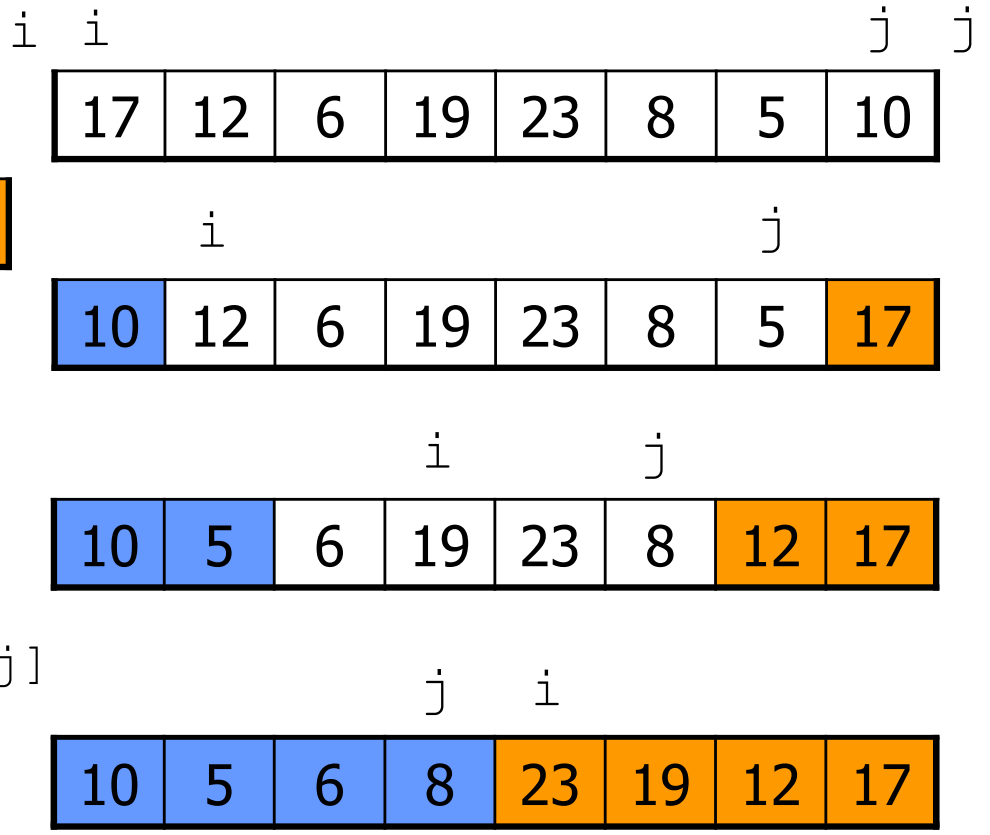


Partição

- Procedimento de partição em tempo linear

```
Partição(A,p,r)
01  x←A[r]
02  i←p-1
03  j←r+1
04  while TRUE
05      repeat j←j-1
06          until A[j] ≤x
07      repeat i←i+1
08          until A[i] ≥x
09      if i<j
10          then Troque A[i]↔A[j]
11          else return j
```





Partição de lomuto

- Método atribuído a Nico Lomuto e popularizado por Bentley (o mesmo do teorema mestre!) e no livro do Cormen.
- Este método escolhe um pivô tipicamente no início ou no final do array. O Particiona recebe como parâmetro dois índices do array.
- Dizem que é mais fácil de entender, mas é menos eficiente principalmente por pivot ser o 1o ou o último.
- Dá pra trocar o 1o ou último com o do meio antes de começar..

Algoritmo Quicksort

- Chamada inicial
  - **Quicksort(A, 1, tamanho de A)**

```
Quicksort(A,p,r)
01  if p<r
02     then q ← Partição(A,p,r)
03             Quicksort(A,p,q)
04             Quicksort(A,q+1,r)
```

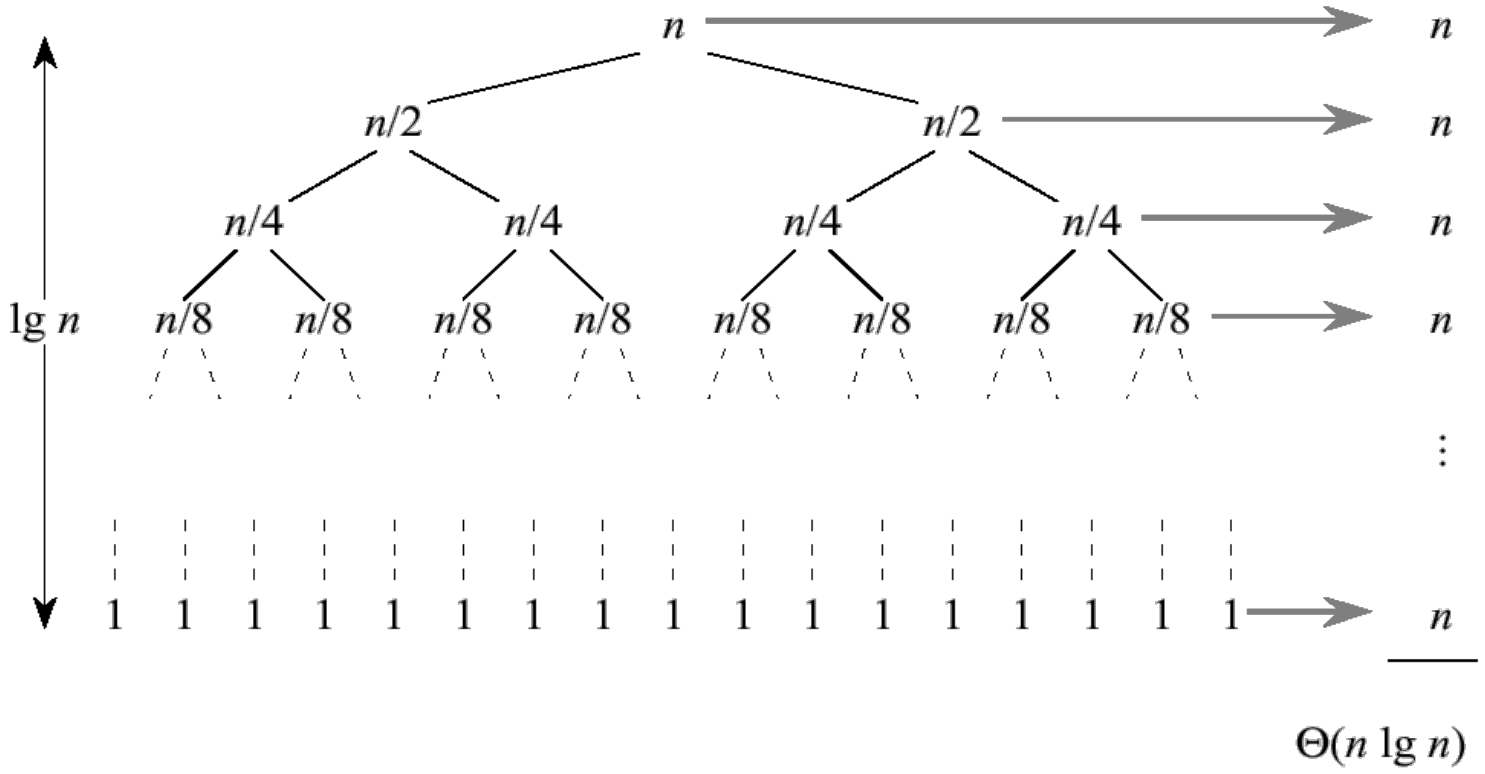
Análise do Quicksort

- Assuma que todos os elementos de entrada são distintos
- O tempo de execução depende de como as divisões ocorrem durante a execução do algoritmo

Análise do QS: Melhor caso

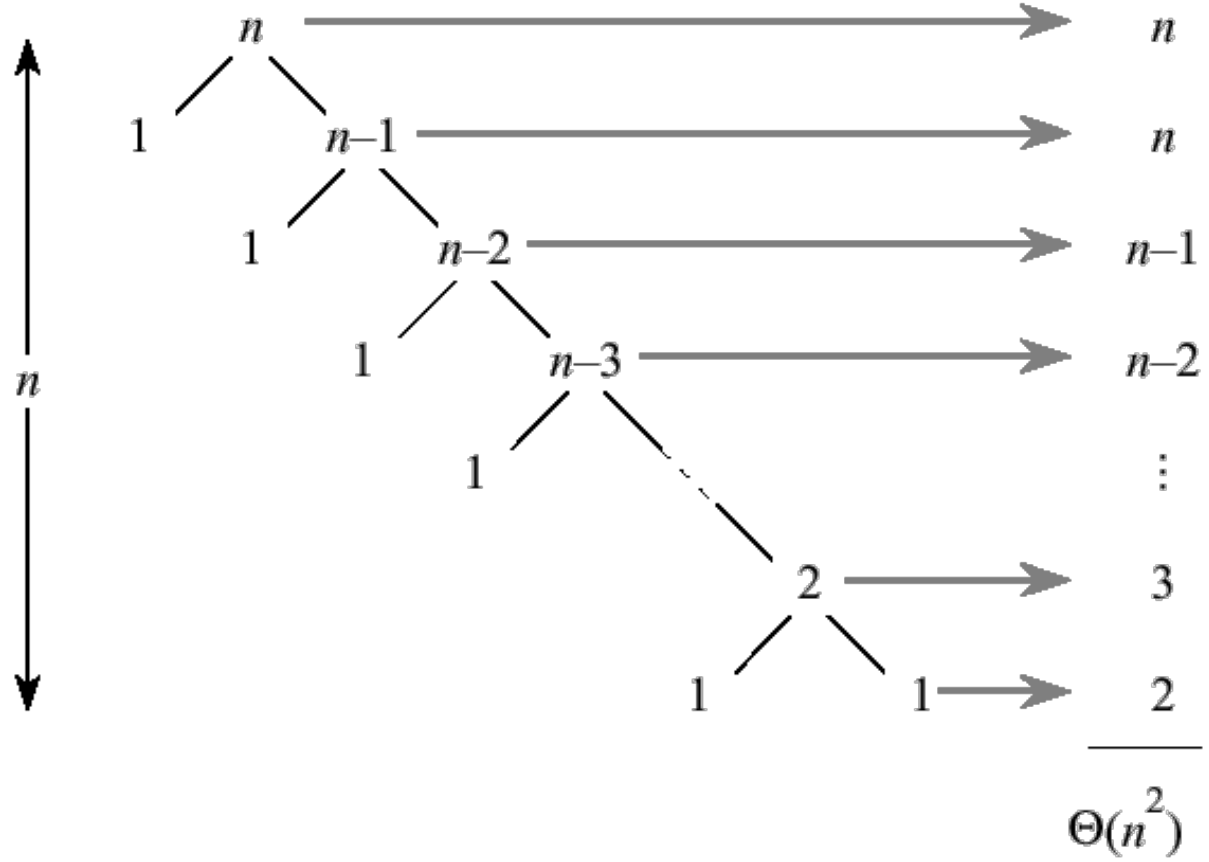
- No melhor caso, as divisões são semelhantes

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$$



Análise do QS: Pior caso

- Um lado da partição fica sempre com um elemento



Análise do QS: Pior caso (2)

$$T(n) = T(1) + T(n-1) + \Theta(n)$$

$$T(n) = T(n-1) + \Theta(n)$$

$$T(n) = T(n-2) + \Theta(n) + \Theta(n)$$

$$T(n) = T(n-k) + \overbrace{\Theta(n) + \ldots + \Theta(n)}^{k}$$

$$T(n) = \Theta\left(\sum_{k=1}^{n} k\right)$$

$$T(n) = \Theta(n^2)$$

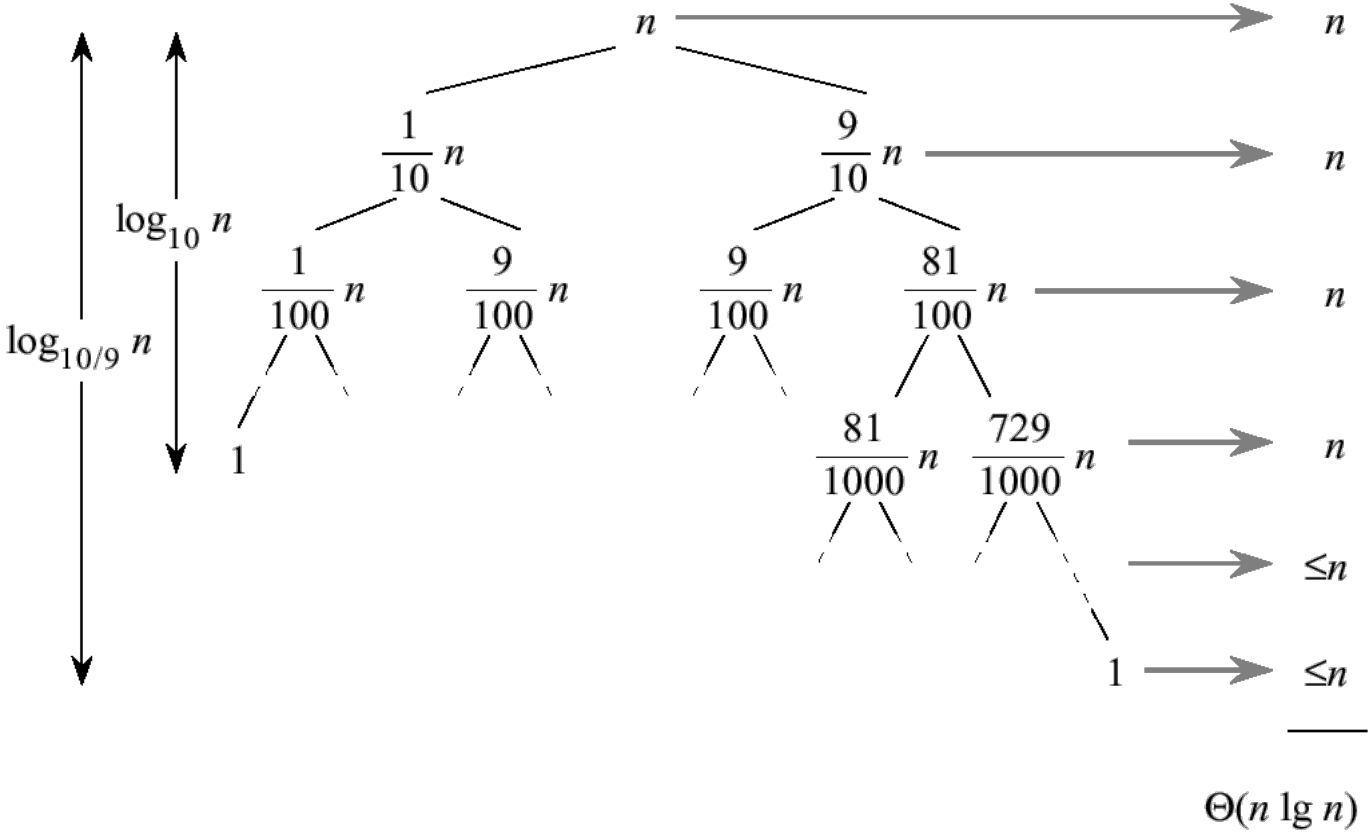
Análise do QS: Pior caso (3)

- Situações onde ocorre o pior caso?
  - A entrada está ordenada
  - A entrada está em ordem inversa
  - Isso é resolvido se pivot for elemento do meio, ao invés do 1º ou do último. Aumenta ligeiramente o custo de achar o pivot, mas melhora em média o desempenho do qsort.

Análise do QS

- Suponha a seguinte divisão 1/10 : 9/10

$$T(n) = T(n/10) + T(9n/10) + \Theta(n) = \Theta(n \log n)!$$



$$\log_{\frac{10}{9}} n = \frac{\log_2 n}{\log_2 \frac{10}{9}}$$

$$\log_{\frac{10}{9}} n = O(\log_2 n)$$

Análise do QS: Caso médio (3)

- Duas hipóteses:
  - Partições são igualmente prováveis
  - Permutações de elementos são uniformemente distribuídas

$$T(n) = T(k) + T(n - k - 1) + c \cdot n$$

$$T_m(n) = \frac{\sum_{k=0}^{n-1} (T_m(k) + T_m(n-k-1) + c \bullet n)}{n}$$

Análise do QS: Caso médio (4)

$$T_m(n) = \frac{\sum_{k=0}^{n-1} (T_m(k) + T_m(n-k-1) + c \bullet n)}{n}$$

$$= cn + \frac{\sum_{k=0}^{n-1} (T_m(k) + T_m(n-k-1))}{n}$$

$$= cn + \frac{2\sum_{k=0}^{n-1} T_m(k)}{n}$$

Análise do QS: Caso médio (5)

$$nT(n) = cn^2 + 2\sum_{k=0}^{n-1} T(k) \quad (1)$$

$$(n-1)T(n-1) = c(n-1)^2 + 2\sum_{k=0}^{n-2} T(k) \quad (2)$$

(1) - (2) $$nT(n) - (n-1)T(n-1) = c(2n-1) + 2T(n-1)$$

$$T(n) = \frac{1}{n}[(2n-1)c + (n+1)T(n-1)]$$

$$T(n) \leq \frac{n+1}{n} T(n-1) + 2c$$

Análise do QS: Caso médio (6)

$$= \frac{n+1}{n} \bullet [\frac{n}{n-1} T(n-2) + 2c] + 2c$$

$$= \frac{n+1}{n-1} T(n-2) + \frac{n+1}{n} 2c + 2c$$

$$= .....$$

$$= \frac{n+1}{2} T(1) + \frac{n+1}{3} 2c + \frac{n+1}{4} 2c + ... \frac{n+1}{n} 2c + 2c$$

$$= O(n) + (n+1)2c(\frac{1}{3} + \frac{1}{4} + ... + \frac{1}{n+1})$$

Análise do QS: Caso médio (7)

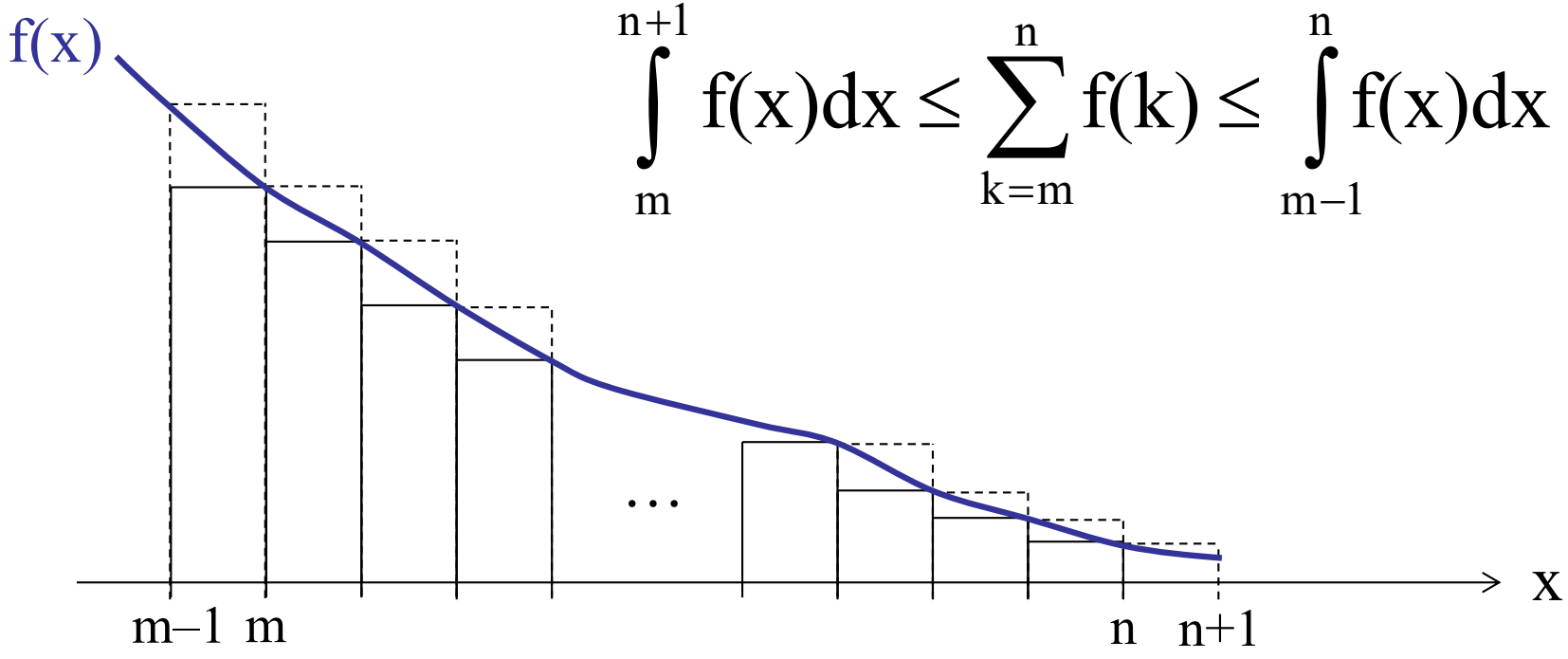
$$\leq O(n) + 2c \bullet (n+1) \bullet H_n = O(n \log n)$$

$$H_n = (1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + ...) \approx \int_1^n \frac{1}{x} dx = \theta(\log n)$$

H_n é a Série Harmônica

Série Harmônica

Aproximação por integrais (livro texto, pag. 842).



Série Harmônica (2)

$$\int_1^{n+1} \frac{dx}{x} \le \sum_{k=1}^{n} \frac{1}{k}, \quad \text{ou} \quad H_n \ge \ln(n+1)$$

$$\sum_{k=2}^{n} \frac{1}{k} \le \int_1^{n} \frac{dx}{x} = \ln n, \quad \text{ou} \quad H_n \le 1 + \ln n$$

Assim,

$$\ln(n+1) \le H_n \le 1 + \ln n$$

Quicksort Randomizado

- Soluções para evitar o pior caso
  - Particionar aproximadamente no meio
  - Escolher um elemento aletório para particionar (funciona bem na prática)
- Algoritmo Randomizado
  - Tempo de execução é independente da ordenação
  - Não há entradas especificas que levam ao pior caso
  - O pior caso é determinado somente pela escolha aletória do elemento de partição
  - Escolher aleatoriamente pode ser mais caro que pegar o elemento do meio.

Quicksort Randomizado (2)

- Assume-se que todos os elementos são distintos
- Particiona-se em torno de um elemento aleatório
- Consequente todas as divisões (1:n-1, 2:n-2, ..., n-1:1) são igualmente possíveis com probabilidade 1/n
- Randomização é uma ferramenta genérica para melhorar algoritmos com pior caso ruim e boa complexidade no caso médio

Quicksort Randomizado (3)

```
Partição-Randomizado(A,p,r)
01  i←Random(p,r)
02  exchange A[r]↔A[i]
03  return Partition(A,p,r)
```

```
Quicksort-Randomizado(A,p,r)
01  if p<r then
02     q←Partição-Randomizado(A,p,r)
03     Quicksort-Randomizado(A,p,q)
04     Quicksort-Randomizado(A,q+1,r)
```

Quicksort – Conclusões

- O Quicksort é, na prática, o algoritmo mais rápido para entradas aleatórias
  - Particionamento coloca os elementos próximos às suas posições finais corretas
- Não é adequado para entradas aproximadamente ordendadas
- Quicksort Randomizado: seleção aleatória do pivot



Curiosidades e melhorias

Partição de lomuto

- Método atribuído a Nico Lomuto e popularizado por Bentley (o mesmo do teorema mestre!) e no livro do Cormen.
- Este método escolhe um pivô tipicamente no início ou no final do array. O Particiona recebe como parâmetro dois índices do array.
- Dizem que é mais fácil de entender, mas é menos eficiente principalmente por pivot ser o 1o ou o último.
- Dá pra trocar o 1o ou último com o do meio antes de começar..
- Há reports na literatura indicando Lomuto é mais lento que Hoare!

Partição

- Vários trabalhos comparam formas de implementar o qsort.
- Em geral, consenso de que Hoare é melhor forma de partição
- Doutorado do Robert Sedgewick (criador das árvores vermelho e preto) foi sobre quicksort

Otimização do Qsort

- Robert Sedgewick propos várias otimizações, principais são:
  - Uso combinado com insertion sort
  - Redução do espaço extra do qsort para log n, fazendo apenas uma chamada recursiva a cada partição.

Melhorando o Qsort



Falando mais em ordenação...

Heap Sort

Williams, J. W. J. (1964), "Algorithm 232 - Heapsort",
Communications of the ACM, 7 (6): 347–348,
doi:10.1145/512274.512284

Heap Sort

- Heap Binário
  - Estrutura de dados implementada usando um arranjo
  - Pode ser vista como uma árvore binária aproximadamente completa
    - Todo os níveis, exceto os inferiores, estão completamente cheios
  - A chave na raiz é maior ou igual que todos os seus filhos
  - As sub-árvores à esquerda e à direita são heaps binários
  - Um Heap *A* tem dois atributos
    - tam-max[*A*]
    - tam-heap[*A*]

Heap Sort (2)

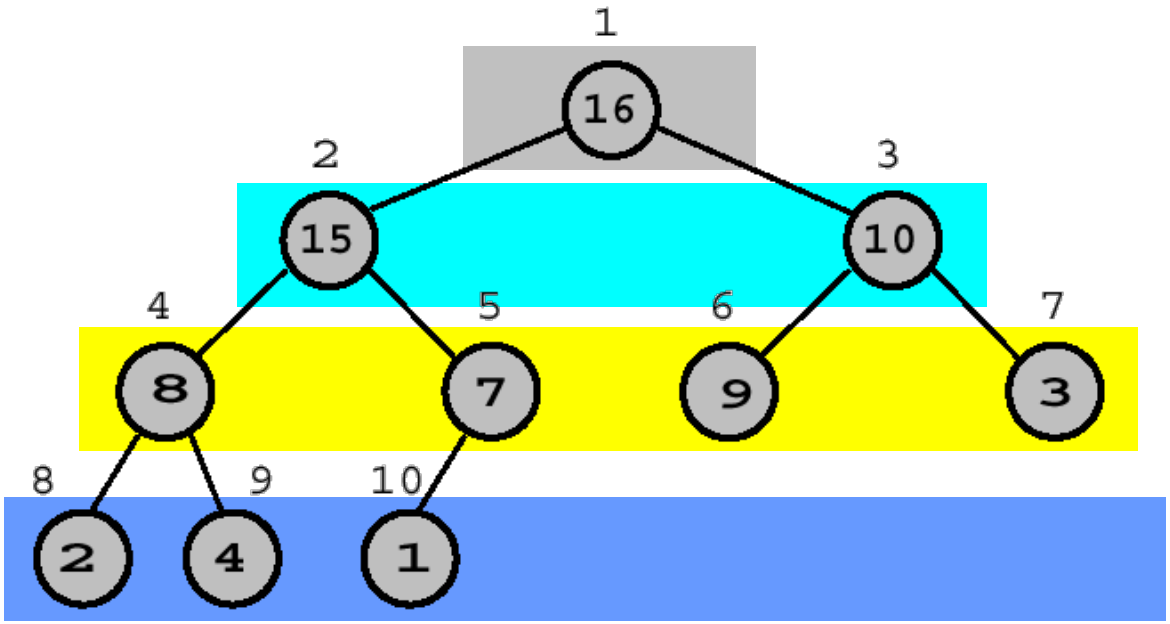
pai (*i*)
retorne $\lfloor i/2 \rfloor$

fesq (*i*)
retorne 2*i*

fdir (*i*)
retorne 2*i*+1

Propriedade:

$A[Pai(i)] \geq A[i]$



1	2	3	4	5	6	7	8	9	10
16	15	10	8	7	9	3	2	4	1

Nível: 3 2 1 0

*Williams, J. W. J. (1964), "Algorithm 232 - Heapsort", Communications of the ACM, 7 (6): 347–348, doi:10.1145/512274.512284

Heapify

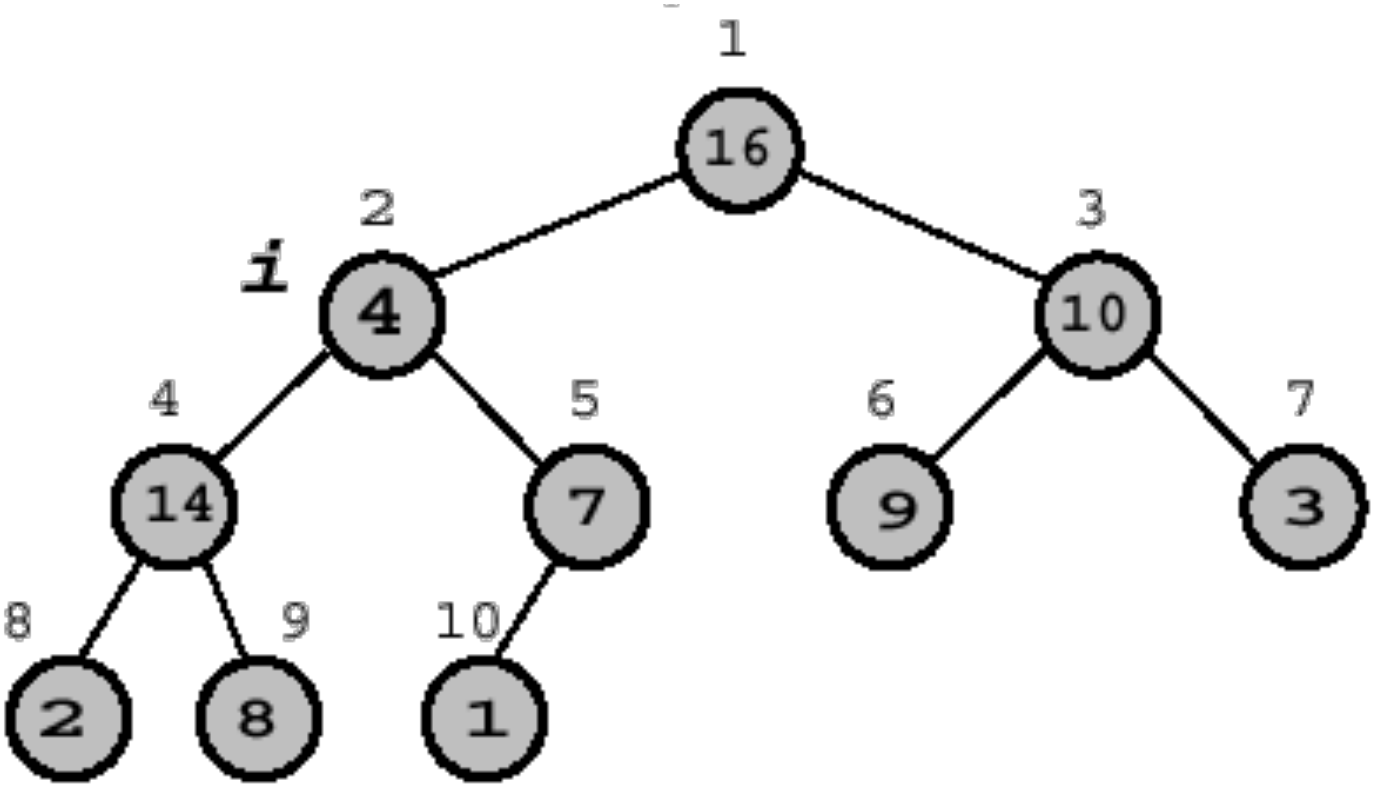
- Seja *i* um índice em um arranjo *A*
- Suponha que as árvores binárias em fesq(*i*) e fdir(*i*) são heaps
- Suponha que *A*[*i*] seja menor que seus filhos, violando portanto a propridade do heap
- O método **Heapify** transforma *A* em um Heap manipulando o elemento em *A*[*i*] até que a propridade de Heap seja satisfeita

Heapify (2)

n is total number of elements

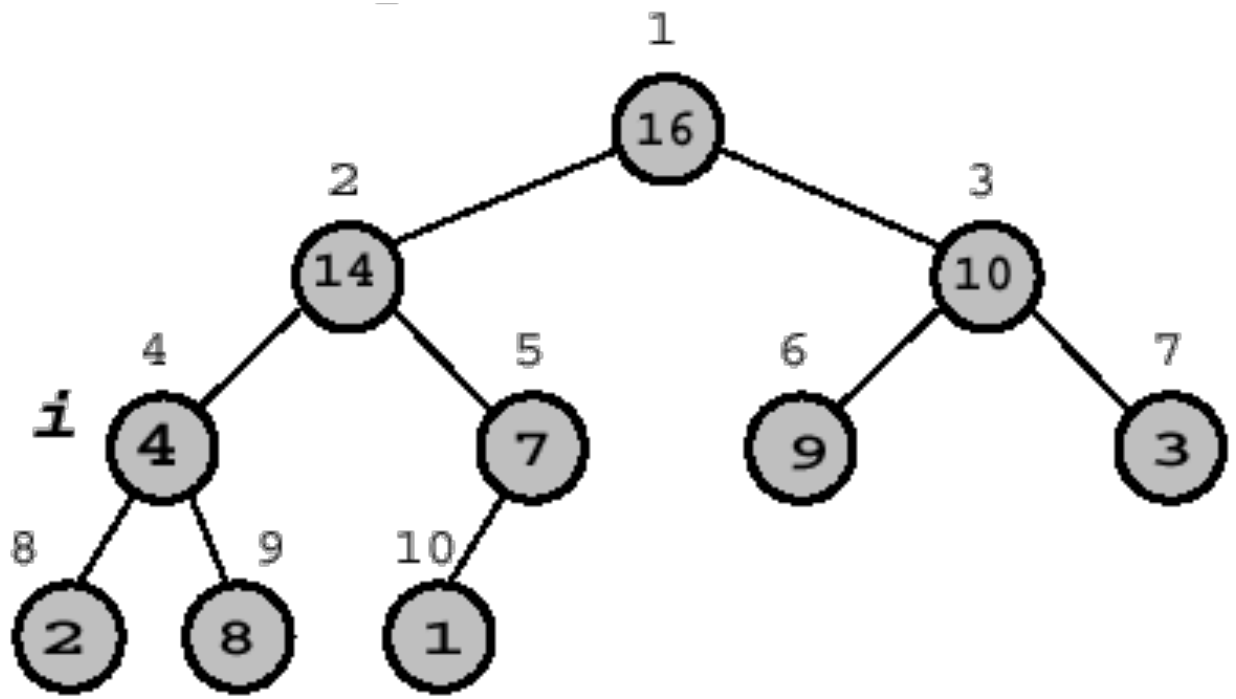
```
HEAPIFY(A, i)
 1 ▷ Left & Right subtrees of i are heaps.
 2 ▷ Makes subtree rooted at i a heap.
 3 l ← LEFT(i)            ▷ l = 2i
 4 r ← RIGHT(i)           ▷ r = 2i + 1
 5 if l ≤ n and A[l] > A[i]
 6     then largest ← l
 7     else  largest ← i
 8 if r ≤ n and A[r] > A[largest]
 9     then largest ← r
10 if largest ≠ i
11     then exchange A[i] ↔ A[largest]
12          HEAPIFY(A, largest)
```

Heapify – Exemplo



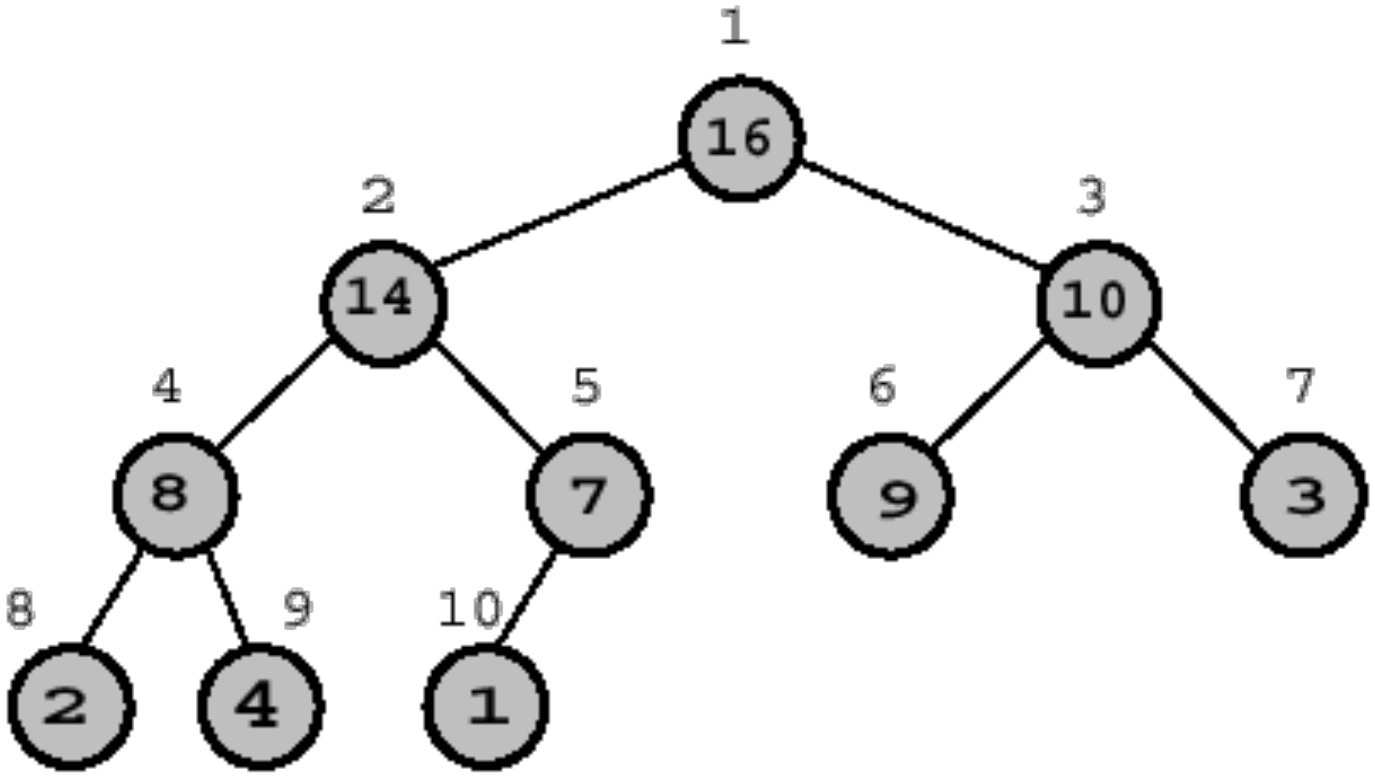
Chamada de Heapify (A,2)

Heapify – Exemplo (2)



Troca de A[2] por A[4] e
chamada recursiva de Heapify (A,4)

Heapify – Exemplo (3)



Troca de A[4] por A[9] e
chamada recursiva de Heapify (A,9)

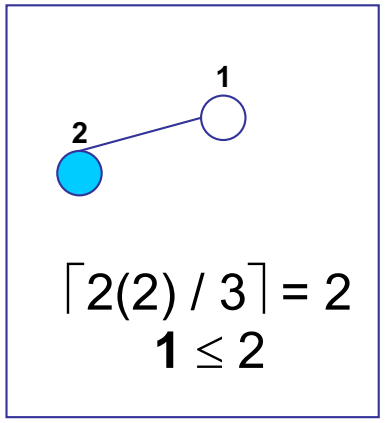
Heapify – Análise

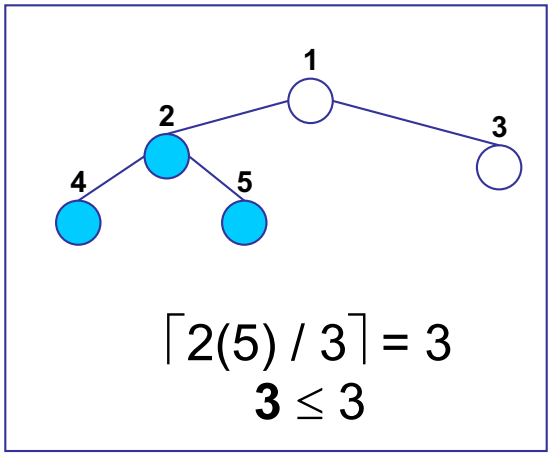
- Tempo de execução de Heapify em uma sub-árvore de tamanho *n* com raiz no nodo *i* é dado por
  - tempo para comparar A[i], A[Left(i)], A[Rigth(i)] : $\Theta(1)$
  - mais o tempo para executar Heapify em uma sub-árvore com raiz em um dos filhos de *i*
  - no pior caso, esta sub-árvore tem 2n/3 nodos

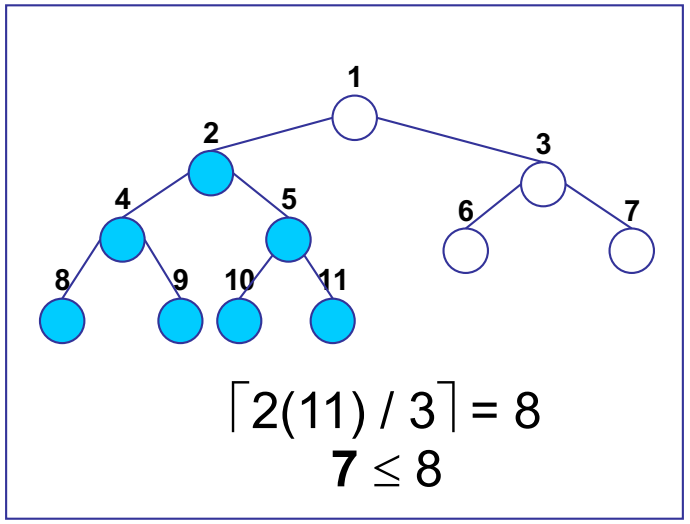
$$T(n) \le T(2n/3) + \Theta(1) \quad \Rightarrow \quad T(n) = O(\log n)$$

Heapify – Pior Caso

- A última linha está metade cheia
- Quantos nodos existem na subárvore mais cheia em termos de n? 2n/3





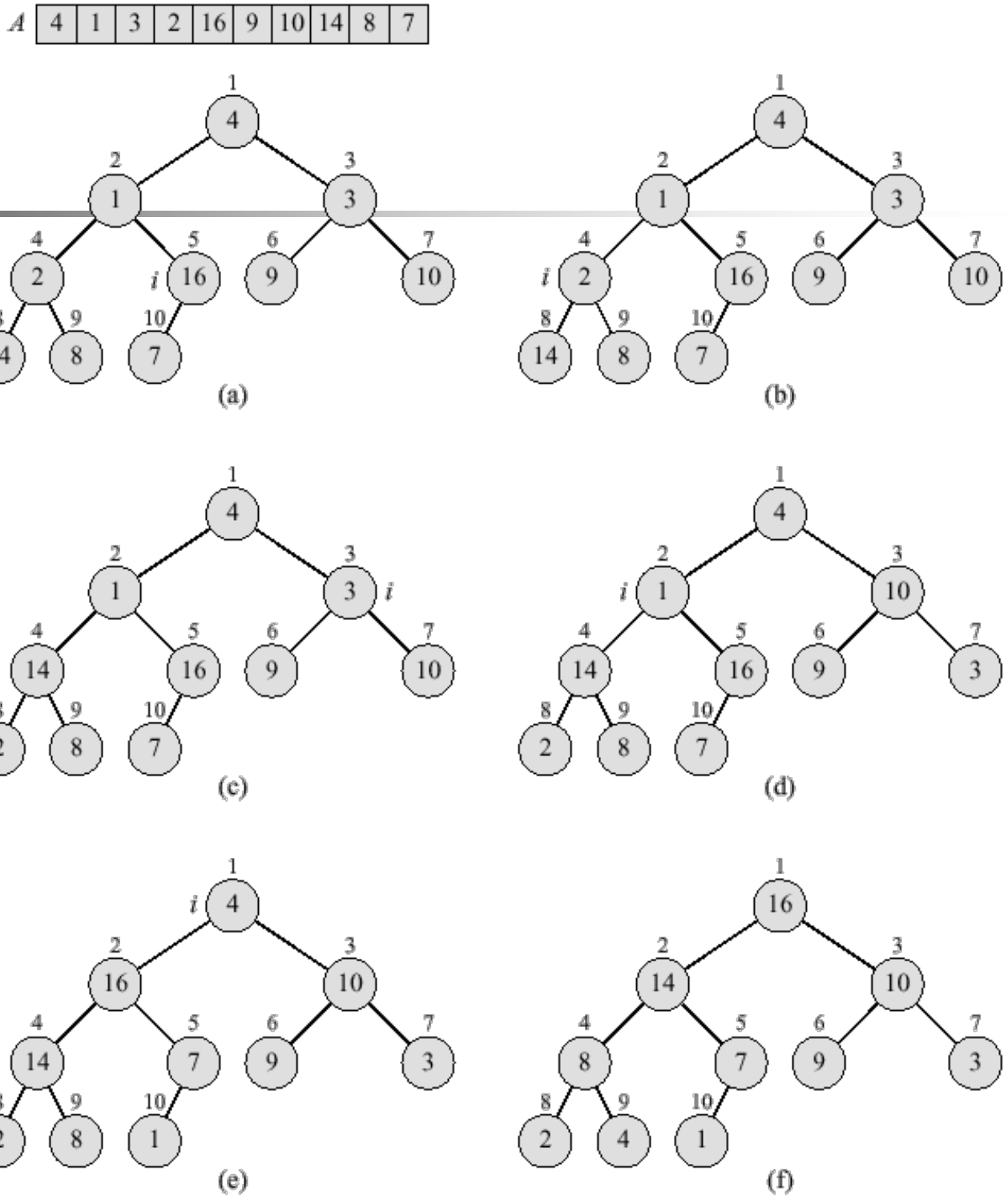


Construção do Heap

- Converter um arranjo $A[1...n]$ em um heap
- Observação importante:
  - Todos os elementos da metade pro fim do arranjo, ou seja $A[(\lfloor n/2 \rfloor + 1)...n]$, são folhas do Heap, ou seja, podem ser considerados Heaps de 1 elemento
  - Assim, basta ”heapificar” a primeira metade!

```
BUILD-HEAP(A)
1 for i ← ⌊n/2⌋ downto 1
2       do HEAPIFY(A, i)
```

Construção do Heap



Construção do Heap – Análise

- Tempo de execução:
  - *n/2* chamadas ao Heapify
  - *n O(lg n) = O(n lg n)*
- Para fins de ordenação *O(n lg n)* é um bom limite assintótico
- Para outras aplicações, é interessante ter um limite mais restrito
  - Intuição: para a maioria dos casos, o Heapify opera sobre menos que *n* elementos do heap

Const. do Heap - Análise (2)

- Definições
  - Altura: caminho mais longo do nodo até as folhas
  - Altura da árvore = Altura da raiz

```
BUILD-HEAP(A)
1 for i ← ⌊n/2⌋ downto 1
2       do HEAPIFY(A, i)
```

  - Tempo pro Heapify = O(altura da sub-árvore de raiz *i*)
  - Suponha $n = 2^k - 1$ (arvore binária completa) $\rightarrow k = \lfloor lg\ n \rfloor$)

$$\sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \log h \rfloor} \frac{h}{2^h} \right)$$

Const. do Heap - Análise (3)

$$\sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^h} \right)$$

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = 2$$

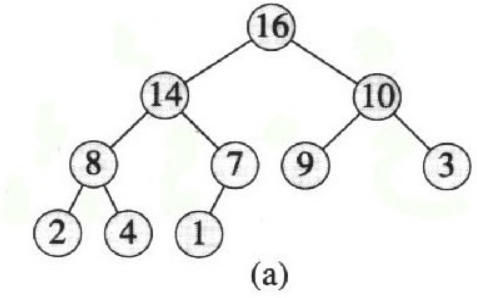
$$O\left(n \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^h} \right) = O(n)$$

Heap Sort

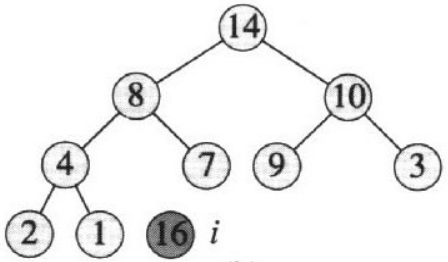
```
HEAPSORT(A)                                   Tempo
 1 BUILD-HEAP(A)                              ? O(n)
 2 for i ← n downto 2                         n times
 3       do exchange A[1] ↔ A[i]              O(1)
 4          n ← n − 1                         O(1)
 5          HEAPIFY(A, 1)                     O(lg n)
```

- Tempo total é *O(n lg n)* + o tempo de Build-Heap(*A*) que é *O(n)*

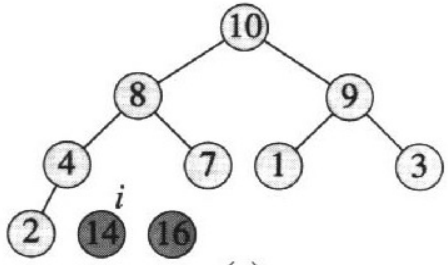
Heap Sort



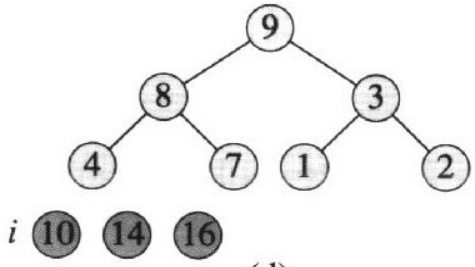
(a)



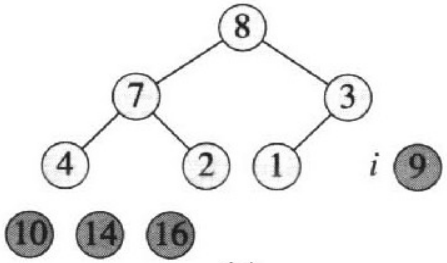
(b)



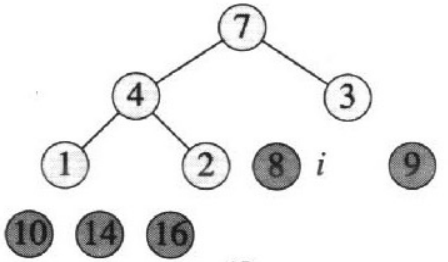
(c)



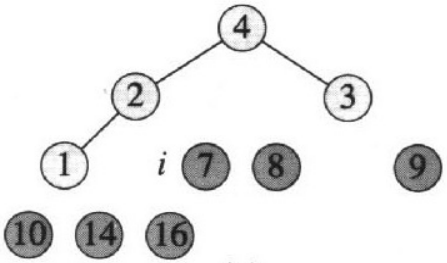
(d)



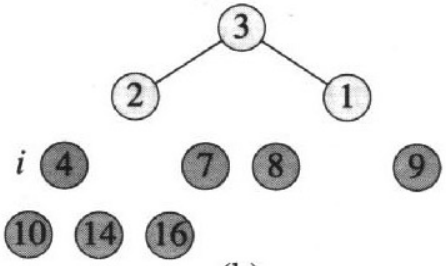
(e)



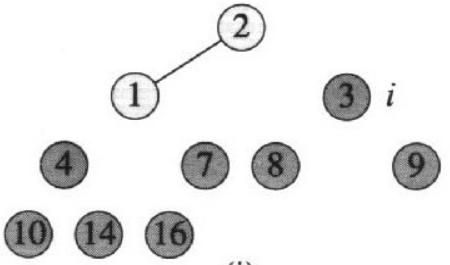
(f)



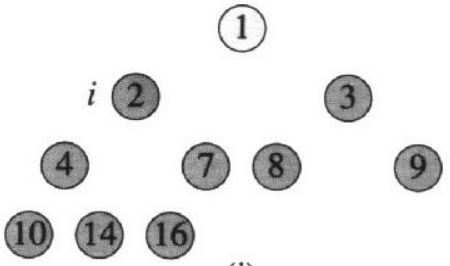
(g)



(h)



(i)



(j)

A	1	2	3	4	7	8	9	10	14	16

(k)

Heap Sort – Conclusões

- O Heap sort usa uma estrutura de dados (Heap) para atingir a complexidade assintótica ótima
- O tempo de execução *O(n log n)* – é o mesmo do merge sort, mas é melhor que o de ordenação por inserção
- Ordenação ”in place” – como o de ordenaçao por inserção e diferente do merge sort

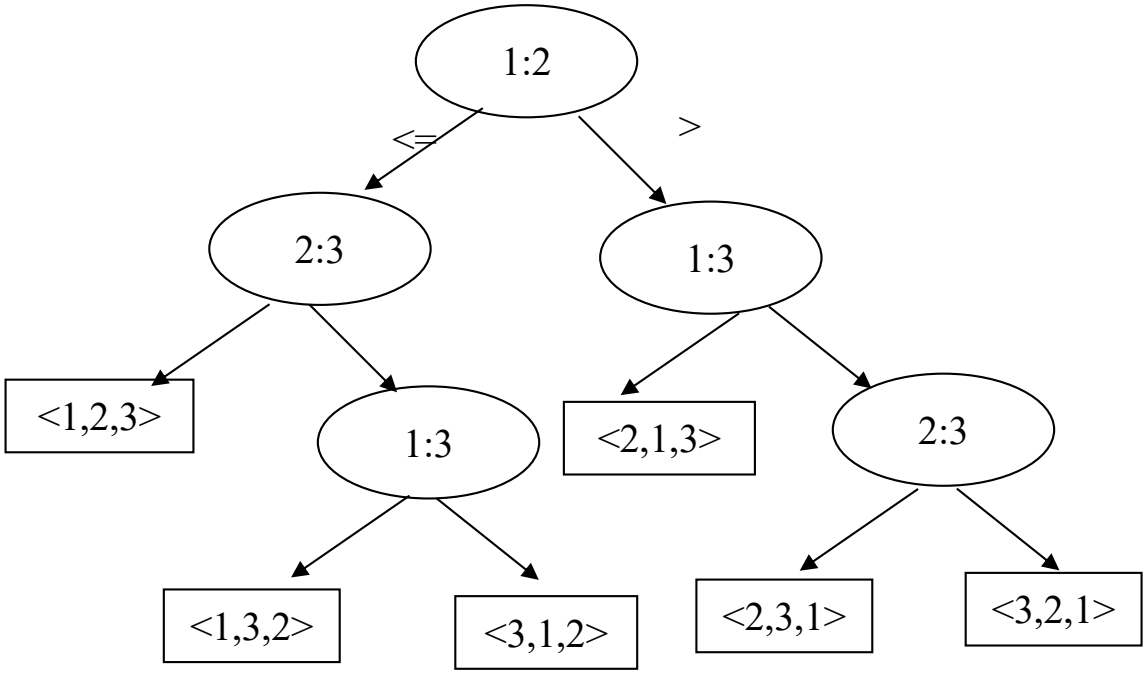


Limite inferior da ordenação

Árvore de decisão

- Árvore binária cheia que representa as comparações executadas por uma algoritmo de ordenação
-

Árvore de decisão



Limite inferior de ordenação

- Árvore de decisão de qualquer algoritmo deve ser capaz de gerar todas as permutações possíveis dos elementos
- Isso porque ele deve mapear qualquer permutação de entrada numa saída ordenada
- Total de permutações é n!
- Limite inferior é o pior caso do melhor algoritmo

Árvore de decisão

- Se o melhor algoritmo produz uma árvore balanceada, então a pior entrada vai ter solução em $\Omega(\log(n!)) = \Omega(n \log(n))$
- Este é o limite inferior para a ordenação!



Mais divisão e conquista: multiplicação de matrizes